

TP Flutter S3 BEAUVALLET : Utilisation de Flutter avec Supabase

Table des matières

1) Problématique.....	1
2) Exercices	1
Partie 1 : Les 4 tutoriels de base avec Supabase.....	1
Partie 2 : Exercices pratiques suggérés sur Supabase	5
3) Conclusion.....	8

1) Problématique

Nous souhaitons savoir comment relier une application Flutter à une base de données en ligne pour afficher et modifier des données, et gérer des utilisateurs. Le sujet de base proposait d'utiliser Firebase, mais nous avons préféré adapter les travaux pratiques avec Supabase pour pouvoir travailler en SQL comme nous avons l'habitude de le faire en cours. Pour cela, nous allons écrire nos fichiers de code avec Notepad++, lancer le serveur de l'application en ligne de commande avec la console Flutter, et vérifier que tout fonctionne bien en ouvrant la page dans Google Chrome.

2) Exercices

Partie 1 : Les 4 tutoriels de base avec Supabase

Exercice 1 : Test de connexion à la base de données

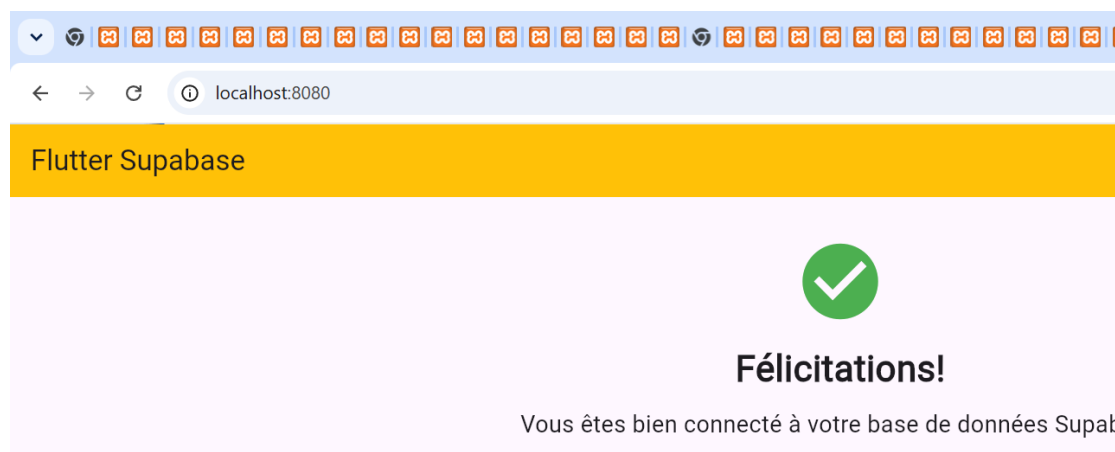
Nous souhaitons d'abord tester si notre application arrive bien à se connecter à Supabase. Pour cela, on ajoute la dépendance dans pubspec.yaml, puis on écrit le code de connexion dans Notepad++ pour aller chercher le texte enregistré dans notre table message.

Le code ressemble à ceci :

```
*c:\Users\Ewan Alexandre\develop\my_app\lib\main.dart - Notepad++
Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

1
2
3 import 'package:flutter/material.dart';
4 import 'package:supabase_flutter/supabase_flutter.dart';
5
6 Future<void> main() async {
7   WidgetsFlutterBinding.ensureInitialized();
8   await Supabase.initialize(
9     url: 'https://xfuxhrznsthtnbvgvms.supabase.co',
10    anonKey: 'sb_publishable_NdJOF1hUhFh1VWbqmMJUg_k_o7MnbW',
11  );
12  runApp(const MyApp());
13 }
14
15 class MyApp extends StatelessWidget {
16   const MyApp({Key? key}) : super(key: key);
17
18   @override
19   Widget build(BuildContext context) {
20     return const MaterialApp(
21       debugShowCheckedModeBanner: false,
22     );
23   }
24 }
```

Comme Notepad++ est juste un éditeur de texte et ne peut pas lancer d'application tout seul, on ouvre une console pour démarrer notre projet avec la commande `flutter run -d web-server --web-port 8080`. On va ensuite sur Google Chrome à l'adresse `http://localhost:8080` et on peut voir que le message de test s'affiche bien :



Exercice 2 : Création de la Todolist en temps réel

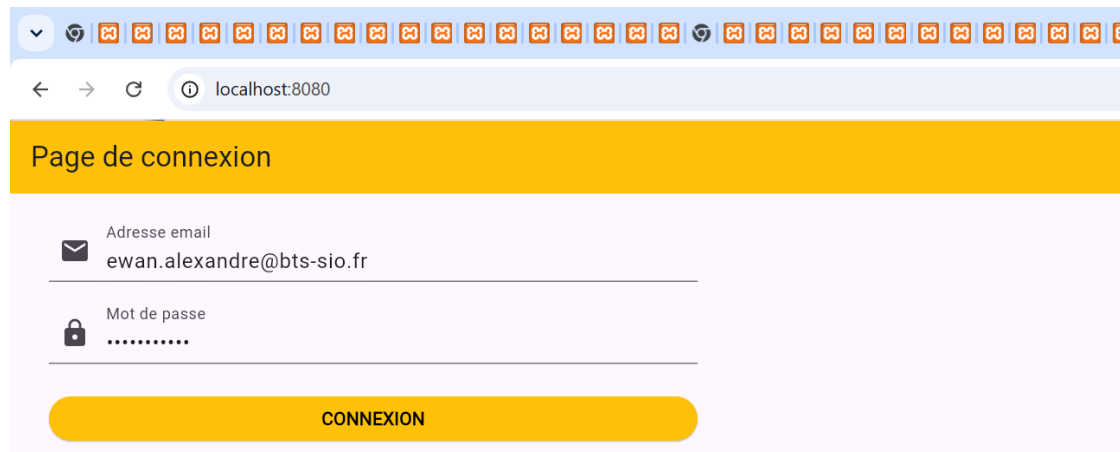
Nous allons maintenant créer une liste de tâches reliée à une table items dans la base. On écrit le code dans Notepad++ pour pouvoir ajouter une tâche avec l'heure, la cocher quand elle est terminée et la supprimer avec une icône de poubelle :

Le code ressemble à ceci :

```
*c:\Users\Ewan Alexandre\develop\my_app\lib\main.dart - Notepad++
Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

1 imp
2
3 import 'package:flutter/material.dart';
4 import 'package:supabase_flutter/supabase_flutter.dart';
5 import 'package:intl/intl.dart';
6
7 Future<void> main() async {
8   WidgetsFlutterBinding.ensureInitialized();
9   await Supabase.initialize(
10    url: 'https://xfuxhrznsthtnbvgvms.supabase.co',
11    anonKey: 'sb_publishable_NdJOF1hUhFh1VWbqmMJUg_k_o7MnbW',
12  );
13  runApp(MyApp());
14 }
15
16 class MyApp extends StatelessWidget {
17   const MyApp({Key? key}) : super(key: key);
18
19   @override
20   Widget build(BuildContext context) {
21     return MaterialApp(
22       debugShowCheckedModeBanner: false,
23     );
24   }
25 }
```

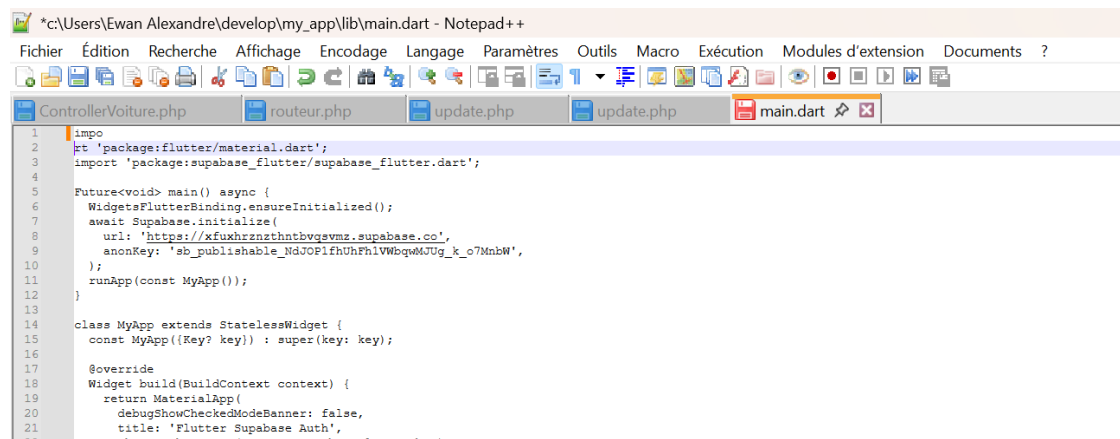
Sur Google Chrome, on peut tester en ajoutant des tâches et en cochant les cases. On voit que la liste s'actualise directement :



Exercice 3 : Inscription, connexion et page de profil

Nous voulons maintenant ajouter un système de connexion pour que l'utilisateur puisse taper son email et son mot de passe, puis être redirigé vers sa page de profil personnel. Voici le code tapé dans Notepad++ :

Le code ressemble à ceci :

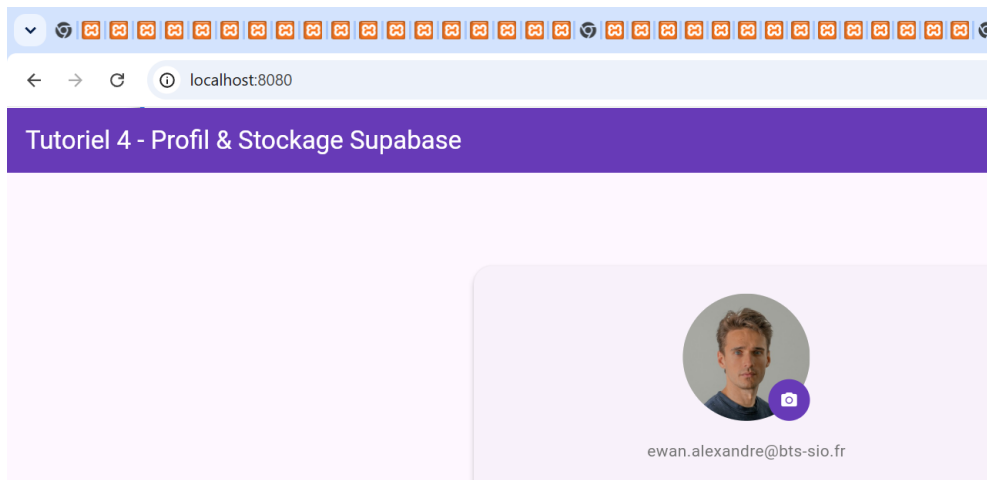


```
1 import
2   'package:flutter/material.dart';
3   import 'package:supabase_flutter/supabase_flutter.dart';
4
5   Future<void> main() async {
6     WidgetsFlutterBinding.ensureInitialized();
7     await Supabase.initialize(
8       url: 'https://xfuxhrznsthtbvgvmz.supabase.co',
9       anonKey: 'sb_publishable_NdJOP1fhUhFh1VWbqm6JUG_k_o7MnbW',
10    );
11    runApp(const MyApp());
12  }
13
14  class MyApp extends StatelessWidget {
15    const MyApp({Key? key}) : super(key: key);
16
17    @override
18    Widget build(BuildContext context) {
19      return MaterialApp(
20        debugShowCheckedModeBanner: false,
21        title: 'Flutter Supabase Auth',
22        theme: ThemeData(
23          colorScheme: ColorScheme.fromSeed(
24            seedColor: Colors.blue,
```

Quand on ouvre la page sur Google Chrome sans être connecté, l'application affiche le formulaire de connexion :



Dès qu'on valide notre compte, on arrive sur la page de profil qui affiche notre email, notre identifiant et nos informations :



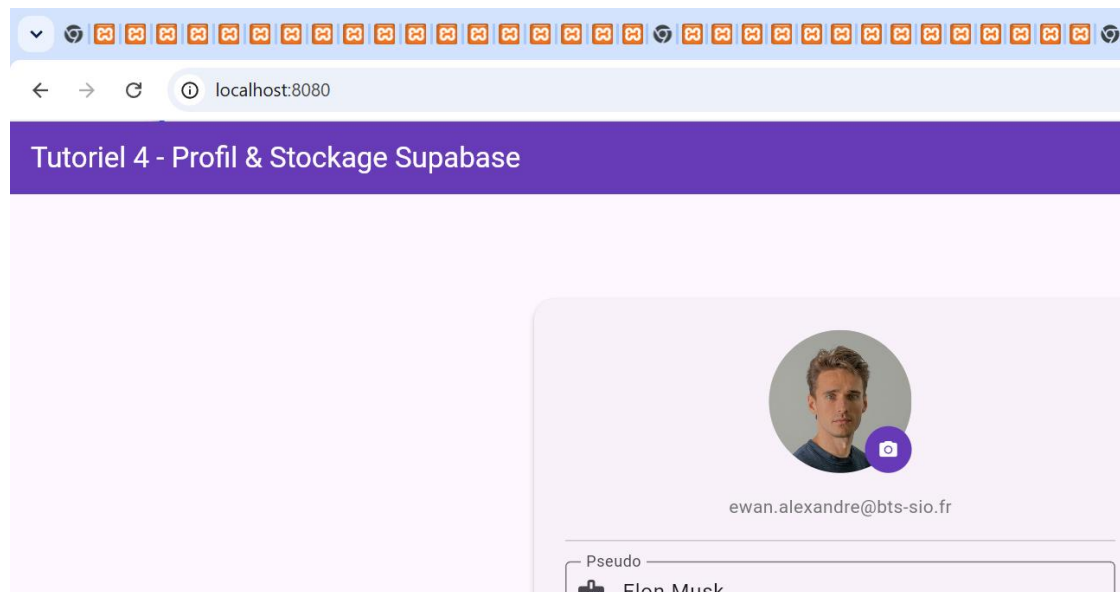
Exercice 4 : Stockage de la photo de profil et informations modifiables

Pour ce quatrième exercice, nous adaptons le stockage de fichiers en utilisant le bucket avatars de Supabase Storage. On permet à l'utilisateur de charger une photo de profil arrondie avec CircleAvatar, et de renseigner son pseudo, sa ville et sa biographie. Voici le code dans Notepad++ :

Le code ressemble à ceci :

```
c:\Users\Ewan Alexandre\develop\my_app\lib\main.dart - Notepad++
Fichier  Édition  Recherche  Affichage  Encodage  Langage  Paramètres  Outils  Macro  Exécution  Modules d'extension  Documents  ?
ControllerVoiture.php  routeur.php  update.php  update.php  main.dart
1  import 'package:flutter/material.dart';
2  import 'package:supabase_flutter/supabase_flutter.dart';
3
4  Future<void> main() async {
5    WidgetsFlutterBinding.ensureInitialized();
6    await Supabase.initialize(
7      url: 'https://xfuxhrznsthtbtbvgevmz.supabase.co',
8      anonKey: 'sb_publishable_Nd3OP1FhUhf1VWbqmMJUg_k_o7MnbW',
9    );
10   runApp(const MyApp());
11 }
12
13 class MyApp extends StatelessWidget {
14   const MyApp({Key? key}) : super(key: key);
15
16   @override
17   Widget build(BuildContext context) {
18     return MaterialApp(
19       debugShowCheckedModeBanner: false,
20       title: 'Tutoriel 4 - Profil & Stockage Supabase',
21       theme: ThemeData(
22         colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
23         useMaterial3: true,
```

Sur Google Chrome, on peut voir la photo de profil avec l'icône de photo, les champs de texte pré-remplis et le bouton pour enregistrer les modifications :



Partie 2 : Exercices pratiques suggérés sur Supabase

Niveaux 1 et 2 : Carnet de contacts CRUD et liaison avec les groupes

Nous voulons créer un carnet de contacts complet relié à deux tables : une table contacts et une table groupes reliées par une clé étrangère groupe_id. Dans l'application, on fait une requête avec jointure pour afficher le nom du groupe à côté de chaque contact. Voici le code dans Notepad++ :

Le code ressemble à ceci :

```

c:\Users\Ewan Alexandre\develop\my_app\lib\main.dart - Notepad++
Fichier  Edition  Recherche  Affichage  Encodage  Langage  Paramètres  Outils  Macro  Exécution  Modules d'extension  Documents  ?
ControllerVoiture.php  routeur.php  update.php  update.php  main.dart

60      await supabase.auth.signInWithPassword(
61          email: 'ewan.alexandre@bts-sio.fr',
62          password: 'password123',
63      );
64  }
65  await chargerGroupes();
66  await chargerStatistiquesBackend();
67  await chargerContactsRelationnels();
68  }
69
70  // Niveau 2 & 5 : Charger les groupes et les statistiques calculées par la fonction backend RPC
71  Future<void> chargerGroupes() async {
72      final res = await supabase.from('groupes').select();
73      setState(() {
74          groupes = List<Map<String, dynamic>>.from(res);
75          if (groupes.isNotEmpty) selectedGroupeId = groupes.first['id'];
76      });
77  }
78
79  Future<void> chargerStatistiquesBackend() async {
80      try {
81          final res = await supabase.rpc('calculer_statistiques');
  
```

Quand on va sur Google Chrome, la liste de nos contacts s'affiche bien avec le badge de leur groupe (Travail BTS SIO, Amis, Famille) et les boutons d'action :



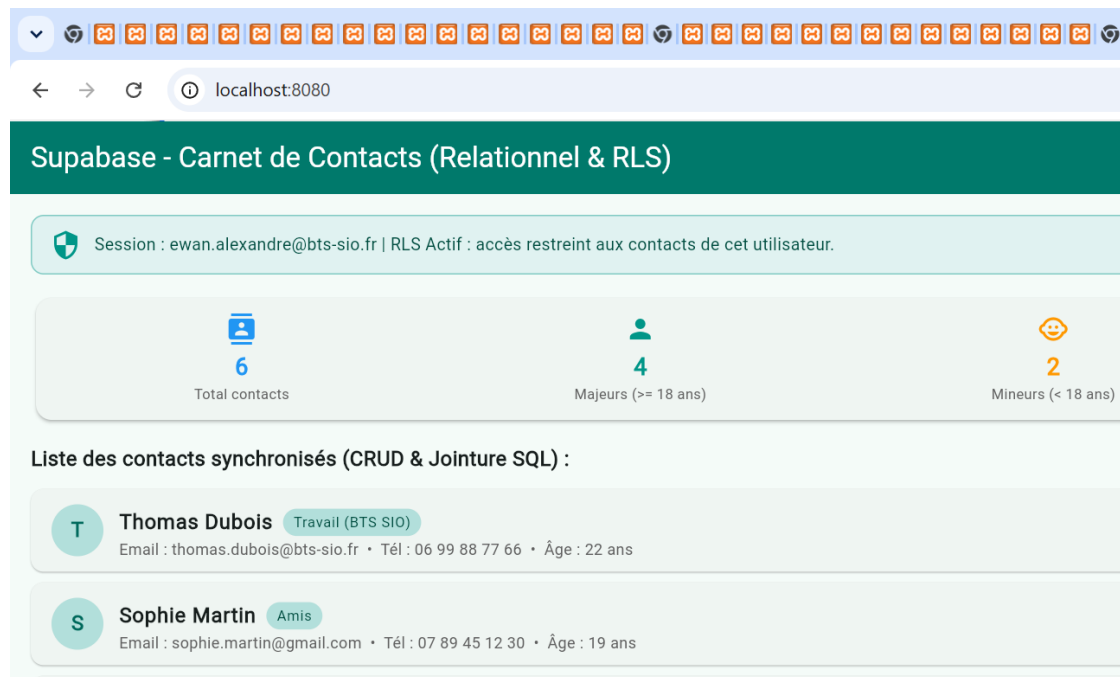
Niveaux 3 et 4 : Sécurité RLS et affichage en temps réel avec .stream()

Pour le niveau 3, nous activons la sécurité RLS (Row Level Security) sur la table contacts pour que chaque utilisateur connecté ne voie que ses propres contacts. Pour le niveau 4, on utilise la fonction .stream() pour écouter la base en continu. Voici le code dans Notepad++ :

Le code ressemble à ceci :

```
*c:\Users\Ewan Alexandre\develop\my_app\lib\main.dart - Notepad++
Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?
ControllerVoiture.php routeur.php update.php update.php main.dart
1 import 'package:flutter/material.dart';
2 import 'package:supabase_flutter/supabase_flutter.dart';
3
4 Future<void> main() async {
5   WidgetsFlutterBinding.ensureInitialized();
6   await Supabase.initialize(
7     url: 'https://xfukhrznsthtbvgvnmz.supabase.co',
8     anonKey: 'sb_publishable_NdJOPfFhUhf1VWbqM4Ug_k_o7MnbW',
9   );
10  runApp(const CarnetContactsApp());
11 }
12
13 class CarnetContactsApp extends StatelessWidget {
14   const CarnetContactsApp({Key? key}) : super(key: key);
15
16   @override
17   Widget build(BuildContext context) {
18     return MaterialApp(
19       debugShowCheckedModeBanner: false,
20       title: 'Supabase - Carnet de Contacts & RLS',
21       theme: ThemeData(
22         colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
23         useMaterial3: true,
24       ),
25     );
26   }
27 }
```

Sur Google Chrome, on voit le bandeau vert indiquant que la sécurité RLS est bien active, et lorsqu'un nouveau contact est ajouté, il apparaît tout de suite sur l'écran sans rafraîchir la page :



Niveau 5 : Calcul de statistiques avec une fonction backend

Pour le niveau 5, plutôt que de faire les calculs dans l'application sur le téléphone, on crée une fonction SQL `calculer_statistiques()` directement sur Supabase qui compte le nombre total de contacts et le nombre de majeurs (âge supérieur ou égal à 18 ans). On l'appelle ensuite avec `supabase.rpc()`. Voici le code dans Notepad++ :

Le code ressemble à ceci :

```
*c:\Users\Ewan Alexandre\develop\my_app\lib\main.dart - Notepad++
Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?
ControllerVoiture.php routeur.php update.php update.php main.dart
1 import 'package:flutter/material.dart';
2 import 'package:supabase_flutter/supabase_flutter.dart';
3
4 Future<void> main() async {
5   WidgetsFlutterBinding.ensureInitialized();
6   await Supabase.initialize(
7     url: 'https://xfukhrznsthtbvgvnmz.supabase.co',
8     anonKey: 'sb_publishable_NdJOPfHhUfHlVWbqMqUg_k_o7MnbW',
9   );
10  runApp(const CarnetContactsApp());
11 }
12
13 class CarnetContactsApp extends StatelessWidget {
14   const CarnetContactsApp({Key? key}) : super(key: key);
15
16   @override
17   Widget build(BuildContext context) {
18     return MaterialApp(
19       debugShowCheckedModeBanner: false,
20       title: 'Supabase - Carnet de Contacts & RLS',
21       theme: ThemeData(
22         colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
23         useMaterial3: true,
24       ),
25     );
26   }
27 }
```

Sur Google Chrome, on peut voir la carte de statistiques au-dessus de la liste avec les valeurs calculées directement par le serveur :



3) Conclusion

En conclusion, ce TP m'a permis d'apprendre à relier une application Flutter à une base de données en ligne comme Supabase. J'ai pu voir comment afficher des données, gérer l'inscription et la connexion d'un utilisateur, stocker des images pour un profil, et faire un carnet de contacts avec des relations SQL et du temps réel. J'ai aussi appris comment lancer le projet depuis la console et vérifier le résultat sur Google Chrome tout en écrivant le code dans Notepad++.